Native Unicode Support in the Unicon Runtime

Jafar Al-Gharaibeh

Unicon Technical Report: 25

August 2026

Abstract

Unicon strings historically have no concept of Unicode: a string is a byte sequence, and operations like subscripting, length, and concatenation operate on bytes. This report describes a native implementation that gives strings UTF-8 awareness while preserving byte-for-byte performance and memory layout for content that is pure ASCII. The approach – tagging a string's existing descriptor rather than introducing a new allocated representation – is the tagged-qualifier design. The report covers the bit-level representation, the mechanisms that apply it, the operations that have been made Unicode-aware, examples, the alternatives that were evaluated and why they were not chosen, and the work that remains. It is intended to remain useful as a reference after the feature has shipped, not only as a plan for building it. Keywords: Unicon, Unicode, UTF-8, strings, runtime, technical report.

Unicon Project
http://unicon.org
University of Idaho
Department of Computer Science
Moscow, ID, 83844, USA

1 1. Introduction

This report records the as-built native Unicode support in the Unicon runtime: how a string is tagged, how literals and I/O become tagged, which operations interpret positions as codepoints, and what is still byte-oriented. It is formatted as a Unicon Technical Report [1]. section 8 collects short programs by operation. Automated tests live under `tests/unicode/`.

The feature is optional. A 64-bit build without `NoUniconUnicode` reports `Unicode` in `&features`. On 32-bit, or with that switch, the tag macros are no-ops and the suite is skipped.

1.1 1.1 Terminology: Unicode vs. UTF-8

These are not interchangeable in this work.

Unicode is the language-facing idea: codepoints, text semantics, and the feature itself (`&features` reports `Unicode`; `_UNICODE`; `UniconUnicode`; `unicode()`, `uchar()`, `uord()`; `*` / `s[i]` on a tagged string). `unicode(s)` means "treat these bytes as Unicode text," not "encode as Unicode."

UTF-8 is the encoding of the payload [2]: the bytes in string space, `open` `i`-mode decoding, lead-byte width scans, and well-formedness. ASCII is a subset of UTF-8; a plain (untagged) string is a byte string. `type()` stays `"string"` for both views. The language-level inverse of `unicode(s)`, and the difference between a content test and a tag test, are discussed in section 7.

Internal C names (`F_UniQual`, `IsUniQual`) stay as the tag; comments should say "UTF-8-tagged qualifier." The class library `UTF8`/`UTF8Set` is correctly named: it is a byte-walking encoding layer. Native support is the Unicode view of those same bytes.

2 2. Design principle and representation

Unicon strings are qualifiers: a two-word descriptor consisting of a length word (`dword`) and a pointer into string space (`vword.sptr`). There is no allocated block and no indirection beyond the pointer itself. This representation is why plain string operations in Unicon are cheap, and it is the property this design is built to preserve: a string containing no non-ASCII

bytes must cost exactly what it costs today, in memory layout, in macro behavior, and in execution speed.

The approach taken is to reuse bits already present in the descriptor's length word rather than introduce a new block type for Unicode-aware strings. A 64-bit word gives far more range than any realistic string length needs, leaving room to encode additional state in the unused high bits without changing the descriptor's size or a plain string's representation at all.

2.1 2.1 Bit layout

Available on 64-bit builds (`WordBits == 64`) when compiled with `UniconUnicode` defined (`src/h/config.h`). With the feature disabled, every macro described below reduces to today's unmodified behavior, so no call site needs its own conditional compilation.

```
bit 63       F_Nqual        (existing -- qualifier vs. typed value)
bit 62       F_UniQual      (this string is UTF-8-tagged)
bits 32-61   cp_count       (30 bits, cached codepoint count)
bits 0-31    byte length    (32 bits, ~4.3 GB maximum)
```

Byte length retains a full 32 bits rather than being reduced further. Unlike the codepoint count, byte length has no fallback: if a string's true length does not fit in its field, there is no secondary source of truth to recover it from, since Unicon strings are not null-terminated. The codepoint count, by contrast, can always be recomputed by walking the string when it is not cached, so it can safely be given whatever bits remain. In practice this is generous rather than tight – the sentinel value (see below) is reached only by a string close to the full 4.3 GB byte-length limit that is almost entirely single-byte content, which would not be UTF-8-tagged in the first place under normal conditions.

Why the remaining bits were spent on a cached codepoint count rather than a cached scan position. Two different kinds of cache were considered for these bits. A cached *codepoint count* records how many codepoints a string contains, making the size operation (`*s`) constant-time. A cached *scan position* instead records a single (codepoint index, byte offset) pair – the last position looked up – letting a subsequent lookup near that position resume without walking from the start of the string. Both are legitimate optimizations, but they help different things: a cached count benefits every

size query uniformly, regardless of string length, since there is no cheaper way to answer "how many codepoints does this string have" without either counting or having already counted. A cached position only pays for itself when a string is subscripted repeatedly and nontrivially; for short strings accessed only a few times, walking from the start is already competitive with or faster than maintaining any index, a result obtained by direct measurement rather than assumed. Because a large share of string values in typical Unicon programs are short-lived tokens – parsed, compared, and discarded, rarely subscripted more than once or twice – the cached-count approach has the stronger claim on the representation's limited bits, and is what this design uses.

A cached-position representation matching the same bit budget was designed and benchmarked in the course of this work (a 14-bit cached codepoint index paired with a 16-bit cached byte offset, packed into the same word) and was found to match a fully allocated index structure to within a few percent for strings up to roughly sixteen thousand codepoints. It was not adopted because the codepoint-count design above has the better-evidenced claim on the same bits, not because it does not work; it remains a candidate for a future representation, discussed in section 9.

2.2 2.2 Core macros

Defined in `src/h/rmacros.h`:

```
StrLen(q)         /* byte length, masked to exclude tag/count bits */
SetStrLen(q,n)    /* sets byte length; see section 2.3 for why this clears
                     rather than preserves the other fields */
IsUniQual(d)      /* tests the UTF-8 tag */
SetUniQual(d)     /* sets the UTF-8 tag */
CpCount(q)        /* reads the cached codepoint count, or the sentinel
                     value 0 if none is cached */
SetCpCount(q,n)   /* sets the cached codepoint count */
```

Three shared helpers implement UTF-8 decoding logic used throughout the runtime:

```
uq_lead_width(b)              /* width in bytes of the UTF-8 character
                                 starting at lead byte b */
```

```
uq_scan(bytes, len, &ncps)    /* scans a byte range once; returns
                                 whether any multi-byte character was
                                 found, and the total codepoint count */
uq_seek_cp(bytes, target_cp)  /* returns the byte offset at which
                                 codepoint index target_cp begins */
```

All three treat a malformed lead byte as a one-byte unit and continue, rather than raising an error; this lenient behavior is specific to internal scanning and is distinct from the strict validation applied by `uord()` (section 4).

2.3 2.3 `StrLen` masking and `SetStrLen` overwrite semantics

`StrLen` unconditionally masks out the tag and count bits before returning a byte length. This has to be unconditional rather than applied only where Unicode-awareness is expected, because a tagged string can reach any code path that handles strings generically – `write()`, list and table operations, anything treating a string as an opaque byte sequence – and any of those reading the raw, unmasked word would compute an incorrect length.

`SetStrLen` performs a full overwrite of the descriptor's length word, rather than clearing only the length bits and preserving whatever tag and count bits were already present. This is a deliberate choice, not an oversight: the large majority of call sites that assign a string length are populating a freshly declared descriptor – a local variable on the stack, an uninitialized table slot – whose prior content is unrelated garbage rather than a meaningful previous value. Preserving that content would leak it into the tag and count fields of what is meant to be a clean, newly constructed string. A full overwrite (with the incoming length itself masked, so that an oversized value cannot bleed into the neighboring count field) produces a deterministic, untagged result in every case, which is what nearly every call site actually requires.

The consequence is that any code that needs to both set a length and retain or apply tagging must do so explicitly, in order: `SetStrLen`, then `SetUniQual`, then `SetCpCount` if applicable. Reversing this order silently loses the tag, since `SetStrLen` clears it. This ordering requirement is not enforced by the type system and must be checked by hand at any call site that

does both. Concatenation (section 5) is the clearest illustration of where getting this wrong produces a real, silent defect: an implementation that computes a combined length via `SetStrLen` and only afterward decides whether to tag the result is correct; one that sets the tag first, relying on an earlier whole-descriptor copy to have carried it, is not, because the subsequent `SetStrLen` call clears whatever the copy carried.

3 3. Promotion: how a string becomes tagged

A string literal's bytes are fixed and known at compile time. Rather than defer classification to when the program runs, the compiler (`icont`) scans a literal once during compilation and encodes the result directly in the compiled program, so that loading and executing the program never needs to re-examine the literal's bytes to determine whether it is Unicode content.

Tagging a literal is a compile-time concern only; there is no runtime scan. An earlier iteration of this work included a runtime scan-on-first-use as a fallback for literals compiled by a version of `icont` predating this feature, on the reasoning that it would let such a program benefit from tagging without recompilation. That reasoning turned out not to hold up: the interpreter cannot distinguish "this literal is untagged because a Unicode-aware compiler determined it is pure ASCII" from "this literal is untagged because the compiler that produced it knew nothing about tagging" – both look identical, an unset tag bit. A scan keyed on "not yet tagged" therefore cannot skip pure-ASCII literals selectively; it must scan every one of them, from every program, on their first execution, to confirm what a Unicode-aware compiler had already determined at compile time. This defeats the purpose of compile-time tagging for the overwhelmingly common case – a program compiled by a current `icont` – in order to support the comparatively rare case of running old, unrecompiled icode. Since recompiling with a current `icont` is a normal, low-cost expectation for picking up a new compiler feature, the runtime scan was removed rather than kept as a fallback. A program compiled by a version of `icont` predating this feature runs with every literal's tag bit unset, which is indistinguishable from, and behaves identically to, this feature being entirely absent – correct, unsurprising behavior requiring no special case, with recompilation being the ordinary path to Unicode-aware literals.

3.1 3.1 Compile-time tagging

`src/icont/tsym.c`'s `putlit`, which registers each literal encountered during compilation, scans a string literal's already-decoded bytes (the compiler's own literal-decoding stage has already resolved any escape sequences by this point, so no further byte transformation occurs between here and the final compiled output) and sets a flag, `F_UniQualLit`, defined in `src/icont/tsym.h` and `src/icont/link.h`. This flag occupies the bit immediately above the four existing literal-type flags (`F_IntLit`, `F_RealLit`, `F_StrLit`, `F_CsetLit`); it is unrelated to the runtime's `F_UniQual` bit position, a separate bit space entirely, and the two should not be conflated.

The flag is carried through the compiler's intermediate representation using the existing mechanism already used for the other literal-type flags, requiring no format changes. At final code emission (`src/icont/lcode.c`'s `uniquallen`), if the flag is set, `F_UniQual` is encoded directly into the length value written into the compiled program. Because this value is written permanently into the compiled program rather than patched in at run time, the interpreter's literal-loading instruction (`Op_Str` in `src/runtime/interp.r`) simply loads it – including on every re-load performed by that instruction's own pre-existing, unrelated self-patching optimization, which rewrites itself after first use to skip recomputing a literal's address on subsequent executions. No interaction between the two was needed beyond this: the tag is already correct in the bytecode the first time it is read, and remains correct on every later read, without any additional mechanism.

One implementation detail is worth recording because it is not obvious from the surrounding code: the values involved in constructing this tagged length are 64 bits wide, while the pre-existing internal representation of a literal's length during compilation is 32 bits (`int`) in two places – the parameter to the function that emits the final instruction, and the field holding a literal's length in the compiler's symbol table. Writing a 64-bit tagged value through either as originally typed would silently truncate it. The chosen fix widens only the function parameter at the point where the tagged value is actually constructed, leaving the symbol-table field's width and the compiler's intermediate file format untouched; the alternative of widening the symbol-table field would have required corresponding changes to that intermediate file's text-based serialization, a larger and riskier change for no additional benefit.

Compile-time tagging encodes the UTF-8 tag but not a cached codepoint

count, since doing so would require the same width change that was deliberately avoided above. As a consequence, a compile-time-tagged literal's codepoint count is not cached until something explicitly computes it; the size operation on such a literal remains correct, falling back to a full scan, but does not receive the constant-time benefit that a cached count would provide until a future revision addresses this (section 9).

4 4. `uchar()` and `uord()`: Unicode-aware character conversion

Unicon's existing `char()` and `ord()` functions convert between an integer ordinal and a single-character string. Both are strictly byte-oriented: `char(i)` accepts only `0 <= i <= 255` and produces a single-byte string; `ord(s)` accepts only a string of exactly one byte. These functions are unchanged by this work and are expected to remain byte-oriented indefinitely, since existing code depends on that behavior for purposes unrelated to Unicode text – base-256 byte encodings, and lookup tables indexed by raw byte value among them.

Two new functions provide Unicode-aware equivalents:

- `uchar(i)` accepts any Unicode scalar value (`0 <= i <= 0x10FFFF`, excluding the surrogate range `0xD800-0xDFFF`, which are not valid standalone Unicode scalar values) and returns its UTF-8 encoding as a string, tagged when the encoding is genuinely multi-byte.

- `uord(s)` accepts a string and returns its codepoint value, requiring that `s` consist of exactly one well-formed UTF-8 encoded codepoint – correct length for its lead byte, correctly formed continuation bytes, no overlong encoding, and no surrogate value. Unlike the lenient byte-width detection used internally for scanning (section 2.2), `uord()` is strict: malformed or wrong-length input raises a runtime error rather than failing silently, matching the convention already used by `ord()` for its own precondition violations.

Both are registered as ordinary built-in functions (`src/h/fdefs.h`) and implemented in `src/runtime/fmisc.r` alongside `char()` and `ord()`. Round-trip correctness (`uord(uchar(i)) = i`) has been verified across both the

Basic Multilingual Plane and the supplementary planes, including four-byte-encoded codepoints.

5 5. Concatenation

The `||` operator (`src/runtime/ocat.r`) has three internal implementation paths, corresponding to different memory-layout optimizations (zero-copy when operands are already adjacent in string space, in-place extension when the left operand is the most recently allocated string, and a general copying path otherwise). All three propagate both the UTF-8 tag and the cached codepoint count to their result: the tag is set if either operand is tagged; the codepoint count is computed as the sum of each operand's contribution, where an untagged operand's own byte length stands in for its codepoint count (valid, since an untagged string is by construction pure ASCII), and a tagged operand contributes its cached count if one is available. If either operand's count is not available (uncached or otherwise unknown), the result's count is left uncached rather than guessed, and a subsequent size query on the result falls back to a full scan.

Per section 2.3, all three paths must apply tagging after computing the result length via `SetStrLen`, not before.

6 6. Deliberately unscanned input: files and sockets

Data arriving through `read()`, `reads()`, and equivalent socket operations is not scanned or tagged automatically, regardless of its actual content. This is a deliberate design decision rather than an oversight, made after comparing a byte-copy-only implementation against one that also scans for non-ASCII content in the same pass. Read operations, unlike literals, have no repeated-execution structure to amortize a scan's cost against: every call processes genuinely new bytes, so a mandatory scan would impose its cost on every call to every program performing I/O, whether or not the data ever contains non-ASCII content. Measured in isolation, a combined copy-and-scan implementation was found to run roughly sixteen times slower than a copy-only implementation at a typical line-read size (128 bytes), and remained an order of magnitude or more slower across read sizes from tens of bytes to several

kilobytes, since a modern `memcpy` implementation is substantially better optimized than an equivalent scanning loop. Given the principle that ASCII content should not pay a cost, automatic scanning of read data was rejected.

This has one consequence worth stating precisely, since it affects the correctness reasoning in section 5: an untagged string is assumed to be pure ASCII by every operation that examines the tag, including concatenation's codepoint-count propagation. That assumption holds for every string this implementation produces internally, but does not hold in general for content read from an external source that happens to contain multi-byte UTF-8 and was never scanned. A program that reads such content and relies on native Unicode-aware operations without first tagging it explicitly may see byte-oriented rather than codepoint-oriented results – correct in the sense that no data is lost or corrupted, but not Unicode-aware until the content is tagged by one of the mechanisms below.

Three mechanisms give a program control over tagging where it is needed, rather than making it automatic:

- `unicode(s)` (`src/runtime/omisc.r`), an explicit, on-demand function. If `s` is untagged, it is scanned; if the scan finds non-ASCII content, the result is tagged and its codepoint count cached. If `s` is already tagged but has no cached count (the common case for a compile-time-tagged literal, section 3.1), the count is computed and cached. If `s` is pure ASCII, it is returned unchanged. `string(s)` is not the inverse; see section 7.

- **The `i` mode character to `open()`**, which sets a per-file or per-connection flag causing subsequent `reads()` calls on that file or connection to scan and tag their results automatically. This is implemented at all of the function's internal return points that produce a taggable result – the file, socket, and secure-shell channel paths – with the exception of pseudo-terminal reads, which return through a different internal mechanism not yet adapted for tagging (section 9).

- **A global default**, changing what `open()` assumes when a caller does not specify a mode explicitly, has been designed but not implemented.

Adding a new file-status flag to `open()`'s mode-character parsing is not sufficient by itself to make the flag usable on every connection type: socket and messaging connections are each additionally validated against an explicit

allowlist of permitted flag combinations (`src/runtime/fsys.r`, `if (status & ~(...)) runerr(.` independent of the character-parsing switch, and a new flag must be added to each relevant allowlist as well or it will be silently rejected for that connection type despite being accepted by `open()`'s general mode parsing.

7 7. Language API: views, conversion, and tests

- `type()` is `"string"` for both views. No `"unicode"` type: `type(x) == "string"` and `||` / `string()` treat tagged values as strings.
- ASCII is never tagged. `unicode("hello")` is a no-op. On ASCII, `*s` / `s[i]` / scanning already match the text view.
- `unicode(s)`: text view (`*` / `[]` are codepoints).
- Inverse of `unicode(s)`: same UTF-8 bytes, untagged descriptor (`*` / `[]` are bytes). Not implemented. Name open. Implementation is a descriptor copy plus `SetStrLen`.
- Current untag path: `writes` to a file, `reads` without `i`. Not an API.

7.1 7.1 `string()` is not the inverse

- `string(x)`: convert to string or fail. Identity on strings, including the tag. `string(a, b, ...)` concatenates and keeps the tag (`||` does the same).
- Must stay identity: `s := string(x) | fail` is the usual coerce. Used in `uni/lib/format.icn`, GUI text fields, XML `string(e)`, IPL. Stripping the tag would make later `*` / `[]` byte-oriented on `"café"`.
- `if string(x) then` still succeeds as a type test if the tag is dropped; `if s := string(x) then` would bind the byte view.
- Rejected inverse names: `bytes(s)` (storage size), `utf8(s)` (encode; opposite of `unicode(s)`), `plain(s)`.
- No builtin `isunicode(s)` if the inverse exists; see section 7.2.

7.2 7.2 Content test vs. tag test

Two different questions. `foo` is the inverse of `unicode`.

Question	isunicode (content)	istagged (descriptor)
Asks	text view ~= byte view?	is `F_UniQual` set on this value?
Form	`*unicode(s) ~= *foo(s)`	must read the bit; no derived form
`*unicode(s)`	codepoints (bytes if ASCII)	
`*foo(s)`	always bytes	

`s`	`*unicode(s)`	`*foo(s)`	content `~=`	tagged?
`"hello"`	5	5	no	no
`"café"` (literal)	4	5	yes	yes
café bytes, no `i`	4	5	yes	no

The last two rows have the same `*` comparison and a different tag. A content test is not a tag test. Inverse plus `*` covers content. A tag predicate is needed only if a program must ask "am I in the text view?" without assigning `unicode(s)`.

8 8. Examples

These examples assume a Unicon build with `Unicode` in `&features`. Non-ASCII literals are tagged at compile time, so `"café"` already has a text view: `*s` and `s[i]` are codepoints. The programs under `tests/unicode/` are the executable form of this section; `languages.icn` is a longer script sample.

8.1 8.1 Length and subscript

`*s` is 4, not 5. `s[4]` is the letter é, not a continuation byte:

```
procedure main()
   local s, i
   s := "café"
   write("*s = ", *s)
   every i := 1 to *s do
      write("s[", i, "] = ", s[i])
end
```

Output:

```
*s = 4
s[1] = c
s[2] = a
s[3] = f
s[4] = é
```

`"hello"` is untagged ASCII, so `*` and `[]` still mean bytes, which for ASCII are the same as codepoints.

8.2 `unicode()`, `uchar()`, and `uord()` ——————————————————————

`unicode(s)` tags non-ASCII UTF-8. `uchar` / `uord` are the codepoint pair; `char` / `ord` stay byte-oriented (section 4):

```
procedure main()
   write(*unicode("hello"))
   write(*unicode("café"))
   write(unicode("café")[4])
   write(uord(uchar(233)))
   write(*uchar(233))
   write(char(65), " ", ord("A"))
end
```

Output:

```
5
4
é
233
1
A 65
```

`uord` on more than one codepoint is error 205.

8.2 8.3 Concatenation

`||` tags the result if either operand is tagged, and adds cached codepoint counts when both are known (section 5):

```
procedure main()
   write(*("café" || "naïve"))
   write(*("café" || "hello"))
   write(*("hello" || "café"))
end
```

Output:

```
9
9
9
```

`string(a, b, ...)` concatenates too and **keeps** the tag, the same as `||`. It is not a byte-view switch (section 7.1).

8.3 8.4 Scanning

On a tagged subject, `move`, `tab`, and `pos` use codepoint positions:

```
procedure main()
   &subject := "café naïve"
   &pos := 1
   write(move(4))
   &pos := 1
   write(tab(5))
   &pos := 1
   write(pos(1))
end
```

Output:

```
café
café
1
```

8.4 8.5 `open()` mode `i`

Without `i`, `reads()` is untagged (byte `*`). With `i`, the same UTF-8 is tagged (section 6):

```
procedure main()
   local f, s
   f := open("uio.dat", "wu") | stop("open")
   writes(f, "café")
   close(f)

   f := open("uio.dat", "r") | stop("open")
   s := reads(f, 100)
   close(f)
   write(*s)

   f := open("uio.dat", "ri") | stop("open")
   s := reads(f, 100)
   close(f)
   write(*s)
   write(s[4])
   remove("uio.dat")
end
```

Output:

```
5
4
é
```

8.5 8.6 Content vs. tag

Untagged UTF-8 from a file, or built from `char()` bytes, has byte `*`. `unicode()` is the text view. The two disagree exactly when the payload is multi-byte (section 7.2):

```
procedure main()
   local raw
   raw := char(195) || char(169)    # UTF-8 of é, untagged
```

```
   write(*raw)
   write(*unicode(raw))
   write(*string(unicode(raw)))
end
```

Output:

```
2
1
1
```

`string(s)` on `"café"` also keeps the tag: `*string(s) = *s`.

9 9. Status and future work

The following operations have been made Unicode-aware. Subscripting (`s[i]`) and section extraction (`s[i:j]`) both include the case of extraction from a named variable, which for either operator is handled by a separate code path (`src/runtime/cnv.r`'s dereferencing of a trapped substring variable) rather than by the operator's own logic directly, and independently applies the same tagging rule – the result is tagged only when the extracted range is genuinely multi-byte, not simply because the source was tagged, since a slice of a tagged string can itself be pure ASCII. Element generation (`!s`) and random selection (`?s`) use the same codepoint unit on a tagged string: each result is one complete UTF-8 character, matching `s[i]`, rather than a single byte that could split a multi-byte encoding. Substring assignment (`s[i] := x` and the range form `s[i:j] := x`) shares this same trapped-variable mechanism on assignment and is complete as well; assigning into a tagged string correctly recomputes the result's tag and codepoint count by scanning the assembled result rather than assuming either the prefix or the suffix inherits the source's tag. `move`, `tab`, and `pos`, which operate on `&subject` and `&pos`, have been made to interpret their positions as codepoints for a tagged subject wherever the underlying position arithmetic (`cvpos`) already treats units abstractly; only the point where an abstract position becomes an actual byte offset needed a tagged-aware branch. `many`, among the pattern-scanning functions, has been made Unicode-aware in one specific respect: it walks a tagged subject codepoint by codepoint rather than byte by byte, so that its notion of

position is codepoint-based. It does not, and cannot yet, test a multi-byte codepoint against the character set argument at all – a plain character set represents only codepoints 0-255, so there is no meaningful membership test to perform for one. The implementation reflects this directly rather than approximating it: on reaching a multi-byte codepoint, it stops immediately, without ever calling the character-set membership test on any of that codepoint's bytes. This was verified deliberately, not assumed – including a check that a character set constructed to contain the exact raw byte values of a multi-byte codepoint's UTF-8 encoding still does not cause the scan to continue through it, ruling out the specific failure mode of accidentally testing individual continuation bytes as if they were independent ASCII characters. Extending `many` (and the character-set type more generally) to test Unicode content meaningfully requires a codepoint-range set type, which does not exist yet (below). `find` and `match`, which search for and test one string against another rather than testing a character set, have also been made Unicode-aware: the byte-level comparison each performs internally is unchanged and does not itself need to become codepoint-aware, since UTF-8 is self-synchronizing – an exact byte match beginning and ending at codepoint boundaries is already a correct codepoint-level match. Only the bounds each accepts and the position each returns needed to change, to mean codepoints rather than bytes for a tagged subject; `find`, as a generator, walks one codepoint at a time across its search window for this reason, testing the byte comparison at each codepoint boundary rather than at each byte offset. The size operator (`*s`), concatenation (`||`), and the explicit `unicode()`, `uchar()`, and `uord()` functions are also complete, as described above.

Table and set key equality, lookup, and membership testing are unaffected by tagging, confirmed by direct test rather than assumed from the general masking behavior described in section 2.3: a tagged literal, a byte-identical untagged string read from a file, and a string explicitly tagged via `unicode()` all compare equal to one another and are interchangeable as table keys and set members, since hashing and equality both operate on `StrLen`-masked byte content only.

`reverse` has also been made Unicode-aware, and needed a different fix than the position-based functions above rather than a lighter version of the same one: a naive byte-for-byte reversal, which is what the original byte-oriented implementation did and is a correct way to reverse an ASCII string, actively corrupts multi-byte content, since reversing byte order breaks the lead-byte/continuation-byte structure a multi-byte encoding depends on.

This was confirmed as real before being fixed, not assumed: reversing a string containing one two-byte character produced a byte sequence a UTF-8 decoder cannot correctly interpret. The fix reverses codepoint order while leaving each codepoint's own bytes in their original internal order, by walking the source forward one codepoint at a time and writing each into the result working backward from the end, so the first codepoint encountered ends up last as a unit rather than having its own bytes individually reversed. Round-trip correctness (`reverse(reverse(s)) = s`, byte for byte) and correct behavior for a string containing several multi-byte codepoints have both been verified.

The following remain byte-oriented and unfixed. `bal`, `any`, and `upto` share `many`'s original, unfixed shape – testing a single position or byte-based run against a character set – and each was confirmed, by direct test rather than by inference from the others, to report byte positions rather than codepoint positions for tagged content: a character immediately following a multi-byte codepoint is reported at its byte offset, one or more past its true codepoint index. For `any` specifically, this was traced further, to the level of individual bytes: testing a position that falls on a multi-byte codepoint's lead or continuation byte tests that isolated byte against the character set independently, rather than recognizing that no plain character set can represent the codepoint as a member at all. `detab`, `entab`, `map`, `trim`, and `sort`/`sortf`, by contrast, were confirmed safe despite being byte-oriented: none of them reorder or split an existing multi-byte byte sequence, so a tagged string's encoding survives each of them intact even though position, padding, or width calculations remain byte-based rather than codepoint-based. `center`, `left`, and `right` fall in this same safe-but-imprecise category. Fixing `bal` and `any` (`upto` and `map` also, since both take a character-set argument) is blocked on the same codepoint-range character-set type as `many`.

Long strings under repeated access. The current implementation performs subscripting on a tagged string by walking from the nearer end of the string on every access; no index or position cache is maintained. This is appropriate for the dominant case in real Unicon programs – short-lived string values accessed a small number of times – but scales poorly for a long string accessed repeatedly, such as a parser scanning across a large source file. Controlled, isolated benchmarking (independent of the runtime, measuring the underlying walk-versus-index algorithms directly rather than the interpreter) found the following, for a 100,000-codepoint string with mixed single- and multi-byte content, comparing the no-cache walk against an indexed scheme combining a single-entry position cache with a bucketed index:

access pattern	walk (no cache)	indexed	ratio
bursty, front-biased (typical tokenizer access)	41.9 us	0.070 us	~600x
sequential scan (typical tokenizing walk)	245.4 us	0.133 us	~1,850x

A block-based representation with an optional index was designed and benchmarked to address this – an allocated structure (tentatively `struct b_unistr`) carrying a single-entry position cache together with an optional bucketed index, with a default bucket size of 8 determined by sweeping the tradeoff between index memory overhead and lookup speed – but was not implemented. The open design question is not the structure itself but the trigger: when a tagged string should be promoted into this representation, and, once promoted, when its index should be upgraded from the cheap single-entry cache to the full bucketed form. Neither threshold was resolved; a length-based threshold was tested directly and found not to work; the more promising direction identified was triggering the upgrade based on observed access cost during actual use rather than on any property known at the time a string is first promoted. A related possibility, not yet implemented, is an explicit optional argument to `unicode()` allowing a program that already knows its own access pattern to request the indexed representation proactively, bypassing the need for an automatic trigger.

A language-level inverse of `unicode()` is still unnamed (section 7). The operation is settled (same bytes, untagged descriptor). `string()` is not that operation. A content test can be derived from the inverse via `*`; a tag test cannot.

`icont` caching a codepoint count, not only the tag, for literals (section 3.1) would remove the one remaining asymmetry between compile-time and runtime tagging, at the cost of the same intermediate-file width consideration that was avoided when implementing the tag itself.

Codepoint-range character sets. Unicon's existing cset type is a 256-bit membership table and cannot represent codepoints beyond 255. A design extending csets to represent arbitrary codepoint ranges, using a sorted range list for codepoints above 255 while leaving the existing bitmap representation untouched for 0-255, was evaluated against alternative representations and found to use substantially less memory across a range of real Unicode block compositions, but was not implemented. `many` (above) is a concrete existing example of what this blocks: it correctly walks a tagged subject by codepoint, but has no character-set representation capable of matching a non-ASCII codepoint against, regardless.

Pseudo-terminal reads do not currently support the `i` mode mechanism described in section 6; the internal function handling them constructs its return value through a different mechanism than the paths that have been adapted, requiring restructuring rather than a direct extension of the existing approach.

Existing UTF-8 support in the standard library. `uni/lib/utf8.icn` [3], a pre-existing class-based UTF-8 implementation predating this work, manages UTF-8 content by walking strings byte-by-byte under the assumption that string indexing is always byte-oriented – a correct assumption before this feature existed, and no longer a safe one for a tagged string, since indexing on a tagged string is codepoint-oriented. The library's tests are consequently excluded from the test suite when this feature is enabled (`tests/general/Makefile`, conditioned on the `unicode` feature being present in `&features`). The library continues to provide functionality with no native equivalent yet – codepoint-range set operations and several scanning primitives – and remains usable by programs that do not enable native tagging; reconciling the two, either by adapting the library to native tagging or by extending native support to cover what it currently provides, is unresolved.

10 Appendix: test coverage

Automated tests specific to this feature live under `tests/unicode/`, one program per concern (`size`, `subsc`, `concat`, `convert`, `scan`, `section`, `bang`, `many`, `match`, `find`, `reverse`, `file_i`, `socket_i`, plus `ascii_*` and `languages`). `tests/general/Makefile` excludes the pre-existing `utf8/utf8_ovld` tests (section 9) when the feature is enabled.

11 References

References

[1] Clinton Jeffery. "How to Write a Unicon Technical Report". doc/utr/utr15.tex. 2013.

[2] Unicode Consortium. "The Unicode Standard". https://www.unicode.org/versions/latest/. 2026.

[3] Unicon Project. "UTF-8 class library". uni/lib/utf8.icn.

[4] The libssh Project. "libssh – The SSH Library". https://www.libssh.org/. 2026.

[5] Clinton Jeffery. "The Unicon Messaging Facilities". doc/utr/utr13.odt.

[6] The OpenSSL Project. "OpenSSL – Cryptography and SSL/TLS Toolkit". https://www.openssl.org/. 2026.

[7] Jafar Al-Gharaibeh. "Native SSH and SFTP Support in Unicon". doc/utr/utr26.rst. 2026.

[8] H. Krawczyk and M. Bellare and R. Canetti. "HMAC: Keyed-Hashing for Message Authentication". RFC 2104. 1997.

[9] S. Deering. "Host Extensions for IP Multicasting". RFC 1112. 1989.

[10] Clinton Jeffery and Shamim Mohamed and Jafar Al-Gharaibeh. "Programming with Unicon". doc/book/. 2026.

[11] Jafar Al-Gharaibeh. "Raw Sockets and Packet Layouts in Unicon". doc/utr/utr29.rst. 2026.

[12] H. Holbrook and B. Cain. "Source-Specific Multicast for IP". RFC 4607. 2006.

[13] Jafar Al-Gharaibeh. "Multicast and Socket Attributes in Unicon". doc/utr/utr28.rst. 2026.

[14] J. Postel. "Internet Control Message Protocol". RFC 792. 1981.

[15] W. Fenner. "Internet Group Management Protocol, Version 2". RFC 2236. 1997.

[16] B. Cain and S. Deering and I. Kouvelas and B. Fenner and A. Thyagarajan. "Internet Group Management Protocol, Version 3". RFC 3376. 2002.

[17] R. Braden and D. Borman and C. Partridge. "Computing the Internet Checksum". RFC 1071. 1988.